

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
```

Em segundo lugar, o servidor Web envia ao cache uma mensagem de resposta com o objeto requisitado:

```
HTTP/1.1 200 OK
Date: Sat, 7 Jul 2007 15:39:29
Server: Apache/1.3.0 (Unix)
Last-Modified: Wed, 4 Jul 2007 09:23:24
Content-Type: image/gif
```

```
(data data data data data ...)
```

O cache passa o objeto ao browser requisitante, mas também o guarda em sua memória cache local. O importante é que ele também guarda, juntamente com o objeto, a data da última modificação. Em terceiro lugar, uma semana depois, um outro browser requisita ao cache o mesmo objeto, que ainda está no cache. Como esse objeto pode ter sido modificado no servidor Web na semana anterior, o browser realiza uma verificação de atualização emitindo um GET condicional. Especificamente, o cache envia:

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
If-modified-since: Wed, 4 Jul 2007 09:23:24
```

Note que o valor da linha de cabeçalho If-modified-since: é exatamente igual ao valor da linha de cabeçalho Last-Modified: que foi enviada pelo servidor há uma semana. Esse GET condicional está dizendo ao servidor para enviar o objeto somente se ele tiver sido modificado desde a data especificada. Suponha que o objeto não tenha sofrido modificação desde 4 Jul 2007 09:23:24. Então, em quarto lugar, o servidor Web envia uma mensagem de resposta ao cache:

```
HTTP/1.1 304 Not Modified
Date: Sat, 14 Jul 2007 15:39:29
Server: Apache/1.3.0 (Unix)
```

```
(corpo de mensagem vazio)
```

Vemos que, em resposta ao GET condicional, o servidor Web ainda envia uma mensagem de resposta, mas não inclui nela o objeto requisitado, o que apenas desperdiçaria largura de banda e aumentaria o tempo de resposta do ponto de vista do usuário, particularmente se o objeto fosse grande. Note que, na linha de estado dessa última mensagem de resposta está inserido 304 Not Modified, que informa ao cache que ele pode seguir adiante e transmitir ao browser requisitante a cópia do objeto que está contida nele.

Finalizamos nossa discussão sobre HTTP, o primeiro protocolo da Internet (um protocolo da camada de aplicação) que estudamos em detalhes. Vimos o formato das mensagens HTTP e as ações tomadas pelo cliente e servidor Web quando essas mensagens são enviadas e recebidas. Também estudamos um pouco da infraestrutura da aplicação da Web, incluindo caches, cookies e banco de dados de apoio, todos associados, de algum modo, ao protocolo HTTP.

2.3 Transferência de arquivo: FTP

Em uma sessão FTP típica, o usuário, sentado à frente de um hospedeiro (o local), quer transferir arquivos de ou para um hospedeiro remoto. Para acessar a conta remota, o usuário deve fornecer uma identificação e uma senha. Após fornecer essas informações de autorização, pode transferir arquivos do sistema local de arquivos para o sistema remoto e vice-versa. Como mostra a Figura 2.14, o usuário interage com o FTP por meio de um agente de usuário FTP. Em primeiro lugar, ele fornece o nome do hospedeiro remoto, o que faz com que o processo cliente FTP do hospedeiro local estabeleça uma conexão TCP com o processo servidor FTP do hospedeiro remoto. O usuário então fornece sua identificação e senha, que são enviadas pela conexão TCP como parte dos

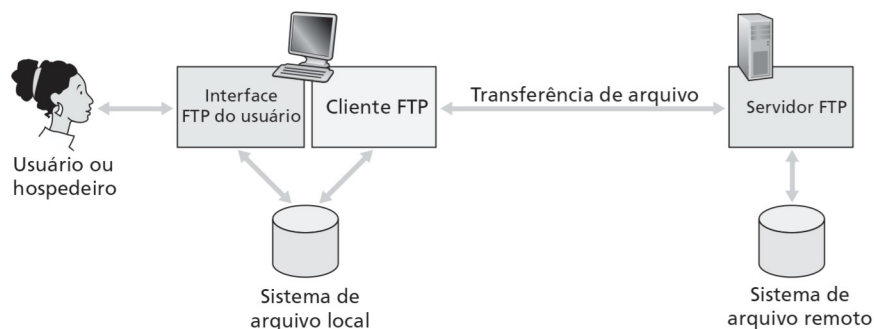


Figura 2.14 FTP transporta arquivos entre sistemas de arquivo local e remoto

comandos FTP. Assim que autorizado pelo servidor, o usuário copia um ou mais arquivos armazenados no sistema de arquivo local para o sistema de arquivo remoto (ou vice-versa).

O HTTP e o FTP são protocolos de transferência de arquivos e têm muitas características em comum; por exemplo, ambos utilizam o TCP. Contudo, esses dois protocolos de camada de aplicação têm algumas diferenças importantes. A mais notável é que o FTP usa duas conexões TCP paralelas para transferir um arquivo: uma **conexão de controle** e uma **conexão de dados**. A primeira é usada para enviar informações de controle entre os dois hospedeiros — como identificação de usuário, senha, comandos para trocar diretório remoto e comandos de ‘inserir’ e ‘pegar’ arquivos. A conexão de dados é a usada para efetivamente enviar um arquivo. Como o FTP usa uma conexão de controle separada, dizemos que ele envia suas informações de controle **fora da banda**. No Capítulo 7, veremos que o protocolo RTSP, usado para controlar a transferência de meios contínuos como áudio e vídeo, também envia suas informações de controle fora da banda. O HTTP, como você recorda, envia linhas de cabeçalho de requisição e de resposta pela mesma conexão TCP que carrega o próprio arquivo transferido. Por essa razão, dizemos que o HTTP envia suas informações de controle **na banda**. Na próxima seção, veremos que o SMTP, o principal protocolo para correio eletrônico, também envia suas informações de controle na banda. As conexões de controle e de dados do FTP estão ilustradas na Figura 2.15.

Quando um usuário inicia uma sessão FTP com um hospedeiro remoto, o lado cliente do FTP (usuário) inicia primeiramente uma conexão TCP de controle com o lado servidor (hospedeiro remoto) na porta número 21 do servidor e envia por essa conexão de controle a identificação e a senha do usuário, além de comandos para mudar o diretório remoto. Quando o lado servidor recebe, pela conexão de controle, um comando para uma transferência de arquivo (de ou para o hospedeiro remoto), abre uma conexão TCP de dados para o lado cliente. O FTP envia exatamente um arquivo pela conexão de dados e em seguida fecha-a. Se, durante a mesma sessão, o usuário quiser transferir outro arquivo, o FTP abrirá outra conexão de dados. Assim, com FTP, a conexão de controle permanece aberta durante toda a sessão do usuário, mas uma nova conexão de dados é criada para cada arquivo transferido dentro de uma sessão (ou seja, a conexão de dados é não persistente).

Durante uma sessão, o servidor FTP deve manter informações de **estado** sobre o usuário. Em particular, o servidor deve associar a conexão de controle com uma conta de usuário específica e também deve monitorar o

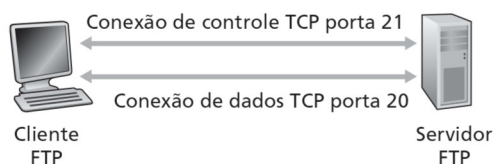


Figura 2.15 Conexões de controle e de dados

diretório corrente do usuário enquanto este passeia pela árvore do diretório remoto. Monitorar essas informações de estado para cada sessão de usuário em curso limita significativamente o número total de sessões que o FTP pode manter simultaneamente. Lembre-se de que o HTTP, por outro lado, é sem estado — não tem de monitorar o estado de nenhum usuário.

2.3.1 Comandos e respostas FTP

Encerraremos esta seção com uma breve discussão sobre alguns dos comandos mais comuns do FTP. Os comandos, do cliente para o servidor, e as respostas, do servidor para o cliente, são enviados por meio da conexão de controle no formato ASCII de 7 bits. Assim, tal como comandos HTTP, comandos FTP também podem ser lidos pelas pessoas. Para separar comandos sucessivos, um ‘carriage return’ e um ‘line feed’ encerram cada um deles. Cada comando é constituído de quatro caracteres ASCII maiúsculos, alguns com argumentos opcionais. Alguns dos comandos mais comuns são descritos a seguir:

USER username: usado para enviar identificação do usuário ao servidor.

PASS password: usado para enviar a senha do usuário ao servidor.

LIST: usado para pedir ao servidor que envie uma lista com todos os arquivos existentes no atual diretório remoto. A lista de arquivos é enviada por meio de uma conexão de dados (nova e não persistente), e não pela conexão TCP de controle.

RETR filename: usado para extrair (isto é, obter) um arquivo do diretório atual do hospedeiro remoto. Ativa o hospedeiro remoto para que abra uma conexão de dados e envie o arquivo requisitado por essa conexão.

STOR filename: usado para armazenar (isto é, inserir) um arquivo no diretório atual do hospedeiro remoto.

Há, particularmente, uma correspondência unívoca entre o comando que o usuário gera e o comando FTP enviado pela conexão de controle. Cada comando é seguido de uma resposta, que é enviada do servidor ao cliente. As respostas são números de três dígitos com uma mensagem opcional após o número. Elas se assemelham, em estrutura, à codificação de estado e à frase da linha de estado da mensagem de resposta HTTP. Os inventores do HTTP incluíram intencionalmente essa similaridade nas mensagens de resposta HTTP. Algumas respostas típicas, junto com suas possíveis mensagens, são as seguintes:

331 Nome de usuário OK, senha requisitada

125 Conexão de dados já aberta; iniciando transferência

425 Não é possível abrir a conexão de dados

452 Erro ao escrever o arquivo

Para saber mais sobre outros comandos e respostas FTP, o leitor interessado pode consultar o RFC 959.

2.4 Correio eletrônico na Internet

O correio eletrônico existe desde o início da Internet. Era uma das aplicações mais populares quando ela ainda estava na infância [Segaller, 1998], e ficou mais e mais elaborado e poderoso ao longo dos anos. É uma das aplicações mais importantes e de maior uso da Internet.

Tal como o correio normal, o e-mail é um meio de comunicação assíncrono — as pessoas enviam e recebem mensagens quando for conveniente para elas, sem ter de estar coordenadas com o horário das outras pessoas. Ao contrário do correio normal, que anda a passos lentos, o correio eletrônico é rápido, fácil de distribuir e barato. O correio eletrônico moderno tem muitas características poderosas. Utilizando listas de mala direta, é possível enviar mensagens de e-mail, desejadas e indesejadas, a milhares de destinatários ao mesmo tempo. As mensagens do correio eletrônico moderno muitas vezes incluem anexos, hiperlinks, textos formatados em HTML e fotos.